

Linear Models 5: Residual Analysis Lab

Dr. Matteo Tanadini

Applied Machine Learning and Predictive Modelling 1, HS25 (HSLU)

Contents

1	Cats Example	2
1.1	Normality of the residuals	3
1.2	Expected value of the errors is “on zero”	5
1.3	Homoscedasticity	7
1.4	Independence of the errors	8
1.5	Influential observations	8
1.6	The plot.lm() function	11
1.7	Further plots (*)	13
2	Appendix	15
2.1	Heteroscedasticity (**)	15
2.2	Tests to test assumptions (**)	16
2.3	Normality of the data	16
3	Session Information	19

This Lab IS NOT to be considered exam material!

This Lab is provided for the sake of completeness. The exam will NOT contain any questions about this Lab.

1 Cats Example

In this exercise we work with the “cats” dataset. Let’s load the data.

```
d.cats <- read.csv("../Datasets/Cats.csv", header = TRUE,
                  stringsAsFactors = TRUE)
##
head(d.cats)
```

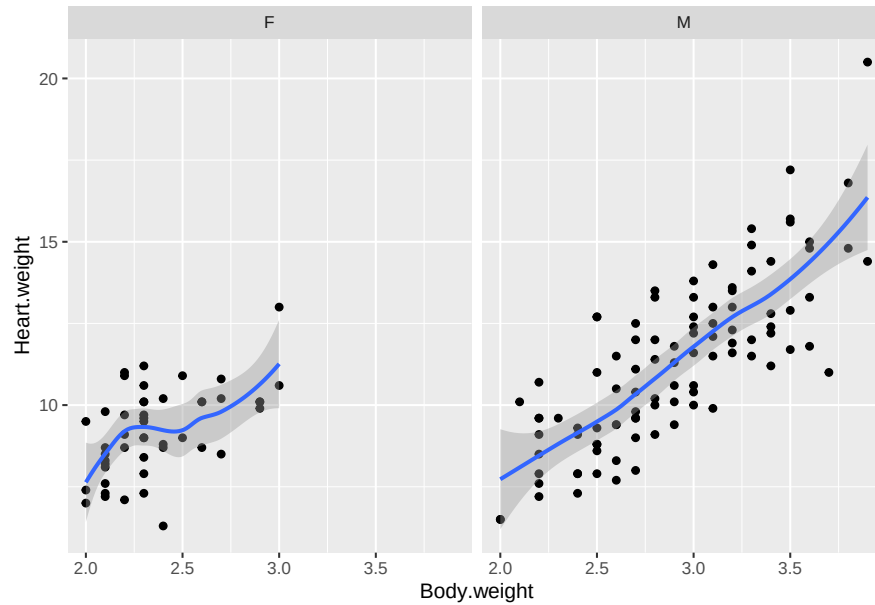
	Sex	Body.weight	Heart.weight
1	F	2.0	7.0
2	F	2.0	7.4
3	F	2.0	9.5
4	F	2.1	7.2
5	F	2.1	7.3
6	F	2.1	7.6

```
str(d.cats)
```

```
'data.frame':  144 obs. of  3 variables:
 $ Sex      : Factor w/ 2 levels "F","M": 1 1 1 1 1 1 1 1 1 1 ...
 $ Body.weight : num  2 2 2 2.1 2.1 2.1 2.1 2.1 2.1 2.1 ...
 $ Heart.weight: num  7 7.4 9.5 7.2 7.3 7.6 8.1 8.2 8.3 8.5 ...
```

We produce a graph to display the data. Note that we use smoothers to characterise the relationship between the response (i.e. heart weight) and the predictor body weight.

```
library(ggplot2)
ggplot(data = d.cats,
       mapping = aes(y = Heart.weight,
                     x = Body.weight)) +
  geom_point() +
  geom_smooth() +
  facet_wrap(. ~ Sex)
```



The effect of *Body.weight* really seems to be linear for males. It is less clear whether assuming a linear effect for females is correct. Assuming a linear effect in both groups, we would expect the slopes to differ.

To start with, we fit a model where we assume that the effect of *Body.weight* is linear, but that can differ between the two gender groups.

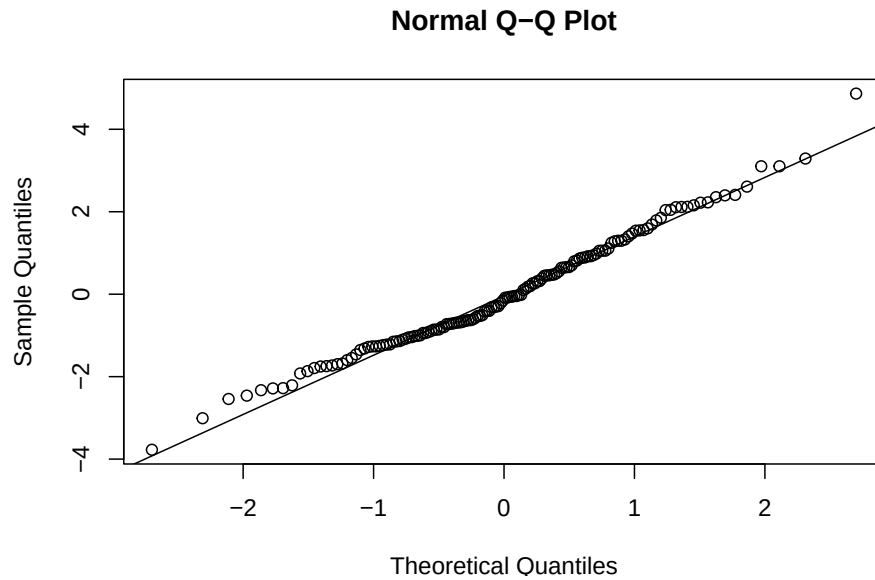
```
lm.cats <- lm(Heart.weight ~ Body.weight * Sex, data = d.cats)
```

Now, before looking at the summary results, we must make sure that the assumptions about the errors are fulfilled. We work with the residuals.

1.1 Normality of the residuals

We first inspect the normality of the residuals of this model.

```
## 1) store the residuals in d.cats
d.cats$resid.lm.cats <- resid(lm.cats)
##
## 2) create the QQ-plot
qqnorm(resid(lm.cats))
qqline(resid(lm.cats))
```



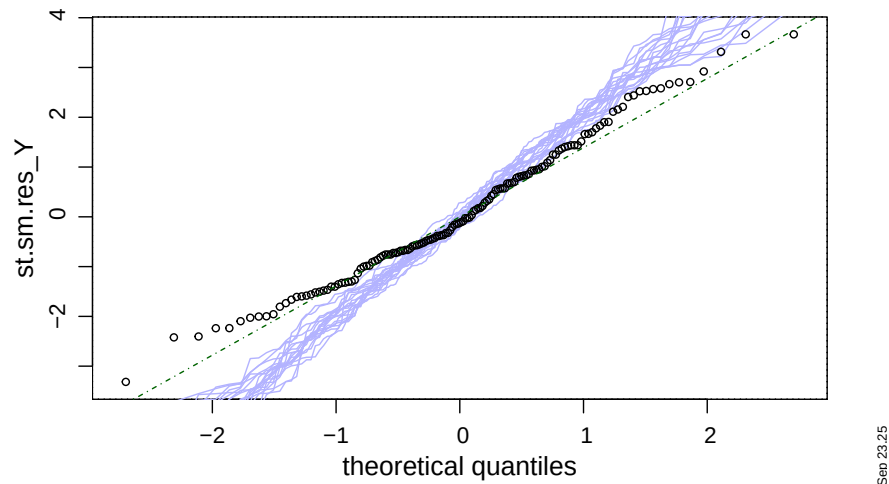
The residuals seem to lay almost perfectly on the reference line. Nevertheless, we may want to use the *{plgraphics}* package that add smoothers to QQ-plots. Note that this package is not on CRAN, where most other packages are stored.

```
## Install the package
install.packages("plgraphics",
  repos = "http://R-forge.R-project.org")
```

Load the package and produce the graph. By using the argument *plotselect* we make clear that we only want the QQ-plot to be produced (the *plregr()* function produces several plots by default).

```
library(plgraphics)
plregr(lm.cats,
  plotselect = c(default = 0,
    qq = 1),
  xvar = FALSE)
```

Heart.weight ~ Body.weight * Sex



This graph confirms that residuals seems to follow a normal distribution. Indeed, the observed observations (in black) lay within the simulated light blue lines.

Note that there would also be some formal tests to test for normality, as explained in the appendix they should not be used.

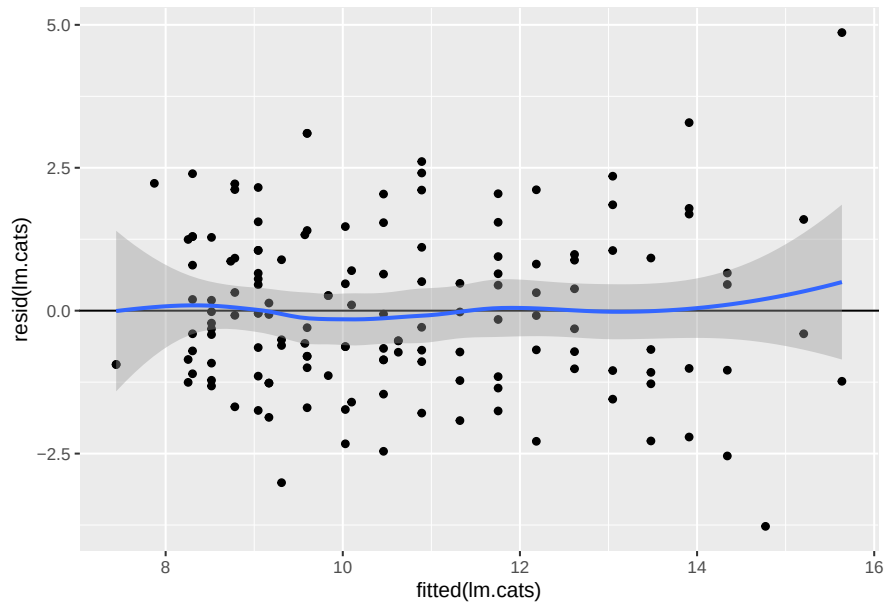
1.2 Expected value of the errors is “on zero”

You may remember from the slides that we tested this assumption by plotting the residuals against the only predictor we had in our model.

In this case there are two predictors (i.e. *Body.weight* and *Sex*). Most often, a model contains many predictors. Therefore, a quick and powerful solution is needed to test the assumption of “error on zero” with one graph only.

The solution to this problem proposed by two famous statisticians (Tukey and Anscombe) consists in plotting the residuals against the fitted values. The plot is nowadays referred as the **Tukey-Anscombe plot** (or TA-plot for short).

```
ggplot(mapping = aes(y = resid(lm.cats),  
                      x = fitted(lm.cats))) +  
  geom_abline(intercept = 0, slope = 0) +  
  geom_point() +  
  geom_smooth()
```

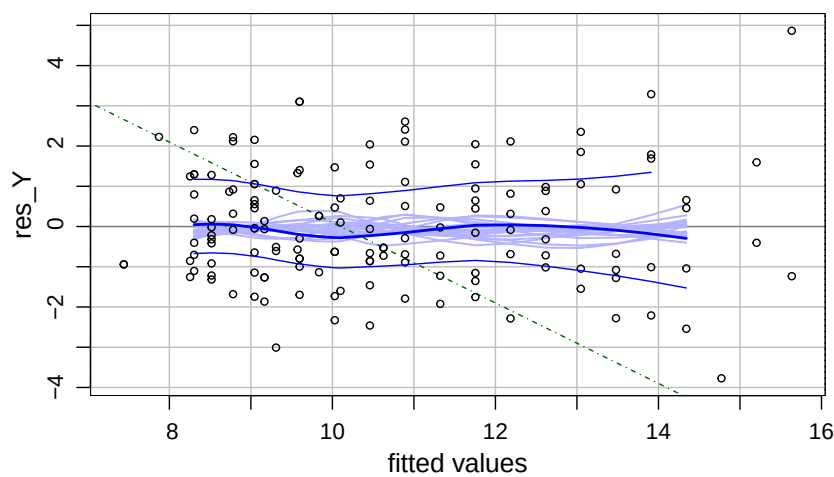


In this case, the smoother is on zero. Therefore, we may conclude that the model equation is correct. If the effect of the *Body.weight* predictor would have been clearly non-linear, then this plot should have given us an hint for that.

We reproduce the same graph with the *plregr()* function.

```
plregr(lm.cats,
  plotselect = c(resfit = 3,
    default = 0,
    absresfit = 0),
  xvar = FALSE)
```

Heart.weight ~ Body.weight * Sex

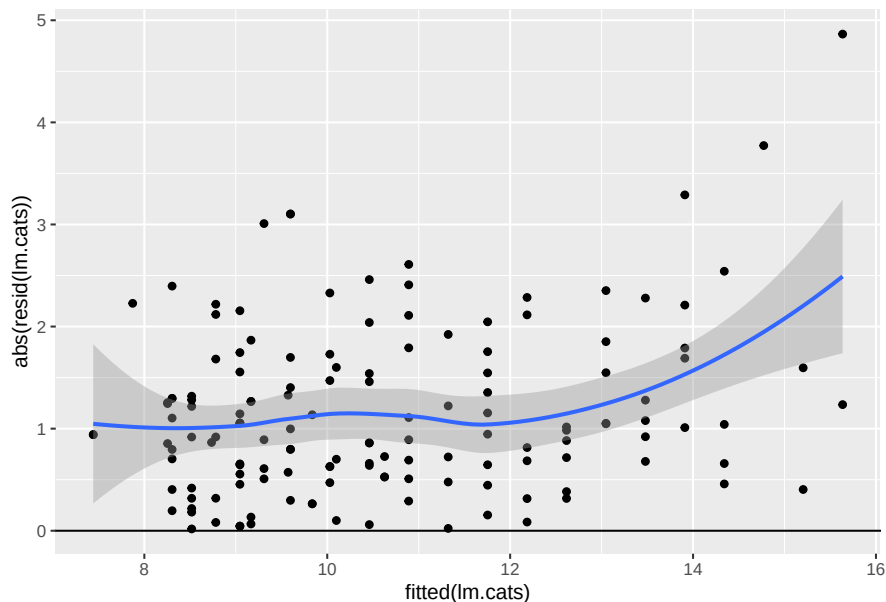


Sep 23, 25

1.3 Homoscedasticity

We now want to control that the “variability” of the residuals is constant. To do that, in analogy with what done just above, we plot the absolute value of the residuals against the fitted values.

```
ggplot(mapping = aes(y = abs(resid(lm.cats)),  
                      x = fitted(lm.cats))) +  
  geom_abline(intercept = 0, slope = 0) +  
  geom_point() +  
  geom_smooth()
```

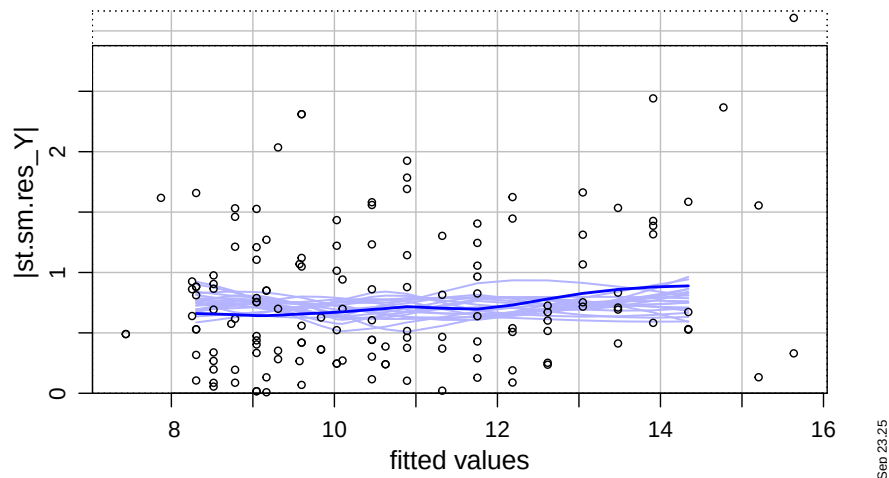


The variance of the residuals seems to be fairly constant apart for the right hand-side of the plot very it seems to increase. Note, however, that this increase may be due a single observation with a very large residual .

We again use a *plgraphics* function to quantify the amount of deviation we may want to tolerate.

```
plregr(lm.cats,  
       plotselect = c(default = 0,  
                       absresfit = 2),  
       xvar = FALSE)
```

Heart.weight ~ Body.weight * Sex



This graph confirm that the residuals seem to have a constant variance. Note that the two graphs are not identical as the latter one is using the “standardised residuals” and a robust smoother (see the *{plgraphics}* documentation for more information).

1.4 Independence of the errors

In this case, we included both predictors available, body weight and gender. Therefore, there is no further information that we may consider in this model. However, sometimes it may turn out that the observations are not totally independent.

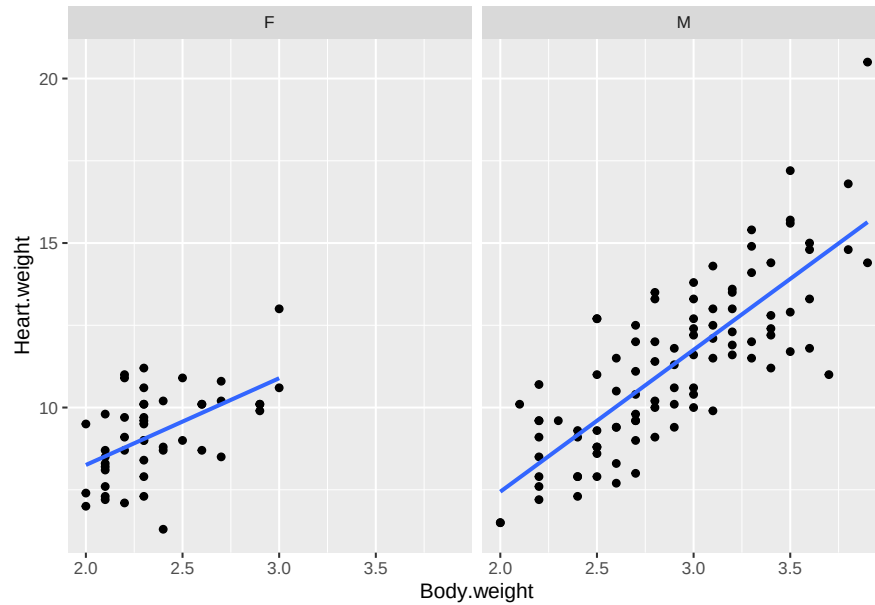
For example, some cats may be siblings. In this case, their observations are likely to be similar. This would a clear breach of the independence assumption. In such a case we may want to include “family” as an additional predictor into the model.

1.5 Influential observations

Let’s reproduce the graph with the raw data to see whether we may have influential observations. Note that we add a regression line to each panel.

```
ggplot(data = d.cats,
       mapping = aes(y = Heart.weight,
                     x = Body.weight)) +
  geom_point() +
  geom_smooth(method = "lm", se = FALSE) +
  facet_wrap(. ~ Sex)
```

‘geom_smooth()’ using formula = ‘y ~ x’



By looking carefully, we see that both females and males have an observation on their right hand-side that has a large positive residual.

To see whether they have a large influence on the estimated regression coefficients we may want to use an influence measure: the *cooks distance*. This quantity measures the effect that omitting a given observation has on the regression coefficients. The highest its value is, the larger the effect.

Here below, we compute the Cook's distance for all observations and then use it to colour observations (not the *colour* argument in the *ggplot()* function call).

```
## 1) compute Cook's distance
cooks.dist.cats <- cooks.distance(lm.cats)
##
## 2) plot it
ggplot(data = d.cats,
       mapping = aes(y = Heart.weight,
                     x = Body.weight,
                     colour = cooks.dist.cats)) +
  geom_point() +
  geom_smooth(method = "lm", se = FALSE) +
  facet_wrap(. ~ Sex)
```

'geom_smooth()' using formula = 'y ~ x'

Warning: The following aesthetics were dropped during statistical transformation:
colour.

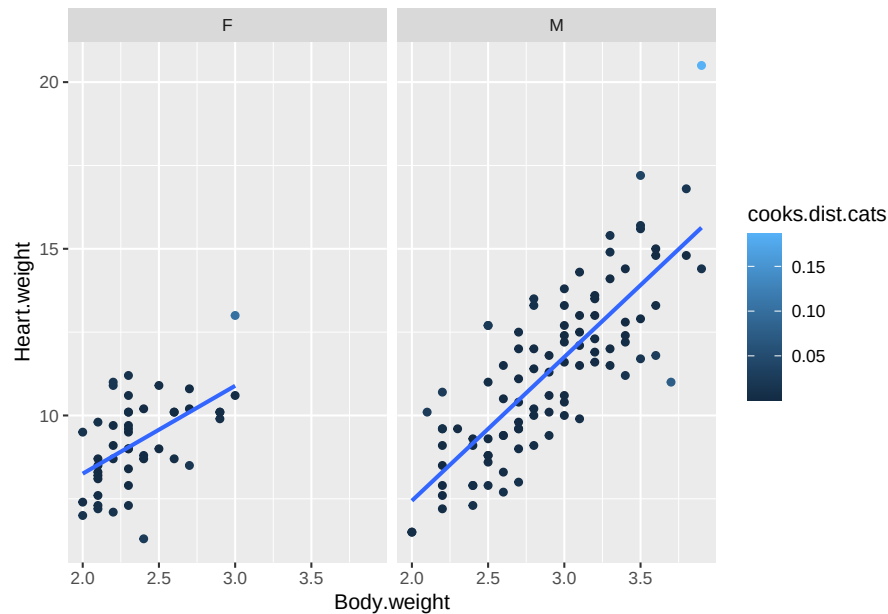
i This can happen when ggplot fails to infer the correct grouping structure in the data.

i Did you forget to specify a 'group' aesthetic or to convert a numerical variable into a factor?

The following aesthetics were dropped during statistical transformation:
colour.

i This can happen when ggplot fails to infer the correct grouping structure in the data.

i Did you forget to specify a ‘group’ aesthetic or to convert a numerical variable into a factor?



As expected, the two observations at the very end of *Body.weight* that have a large residual have also high leverage. The question that raises naturally here, is “when is the Cook’s distance too large? When shall I do something?”

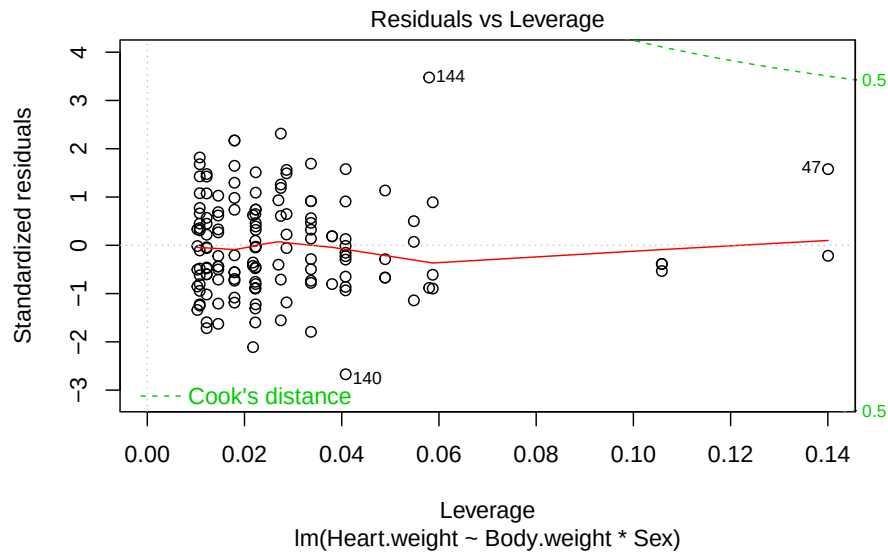
The thumb rule says that observations with a value larger than 0.5 “should be inspected”. Observations that have a value larger than 1 are seriously influencing the estimated regression coefficients and “action should be undertaken”. What exactly should be done with these influential observation is case-specific:

- If the observation turns out to be a mistake, and the correct value cannot be retrieved, the observation can be safely be omitted (but this should be documented).
- If the observation has a logical explanation related to a further predictor not yet included in the model, this latter may be included in the model. For example, the cats with influential observation may be very old cats and, therefore, cat’s age could be included.
- If there is no explanation, and the observation seems to be correct, *robust methods* could/should be used¹.
- If there is no explanation, and the observation seems to be correct, it could be removed. Note, however, that this should be documented and discussed when reporting the results. Furthermore, it is very often the case, that by removing an influential observation and refitting the model, the next influential observation pops up. In this case, it is a problem of long-tailed distribution of the errors, rather than a few “wrong” observations.
- Note that in some cases, a missed non-linearity can also lead to influential observations. In theory, we would have noticed in the preliminary graphical analysis that a given predictor may have a non-linear effect. However, sometimes a non-linearity may be overlooked. One solution to spot non-linearities after fitting a model is to plot the residuals against the continuous predictors.

¹See for example the function *lmrob()* in the *{robustbase}* package.

Note that in this simple case, where we have only two predictors in the model, we can display the information about the Cook's distance graphically with the raw data. However, in reality we often have many predictors. Fortunately, there is a built-in **R** function that creates an adequate graph to inspect influential observations.

```
plot(lm.cats, which = 5)
```



On this graph, the standardised residuals are plotted against the leverage measure².

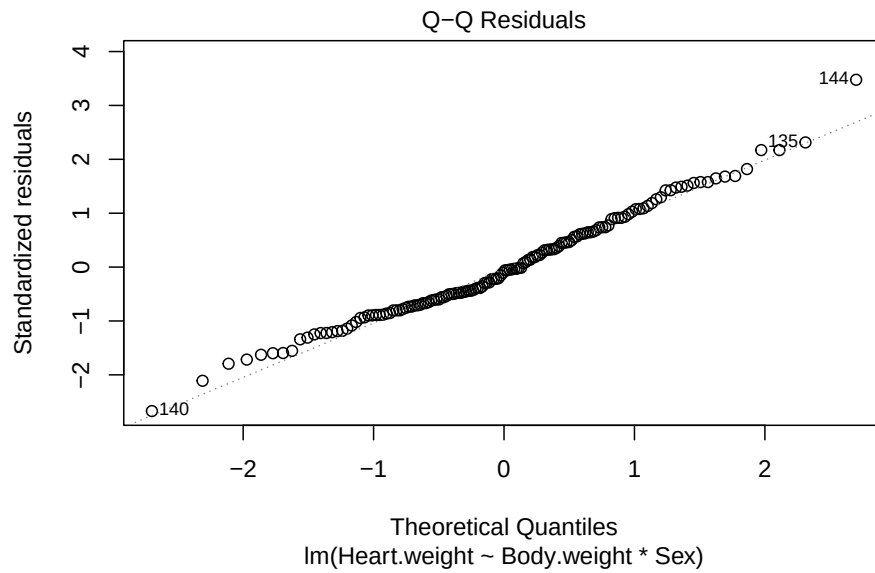
The dashed red lines on the graph, are contour lines for a Cook's distance of 0.5. For the model *lm.males* none of the observations lay outside these boundaries and, therefore, no action is required. The *plot.lm()* function also plots a contour line for a Cook's distance of 1. This line lays outside of the plotting region in this case and is, therefore, not visible.

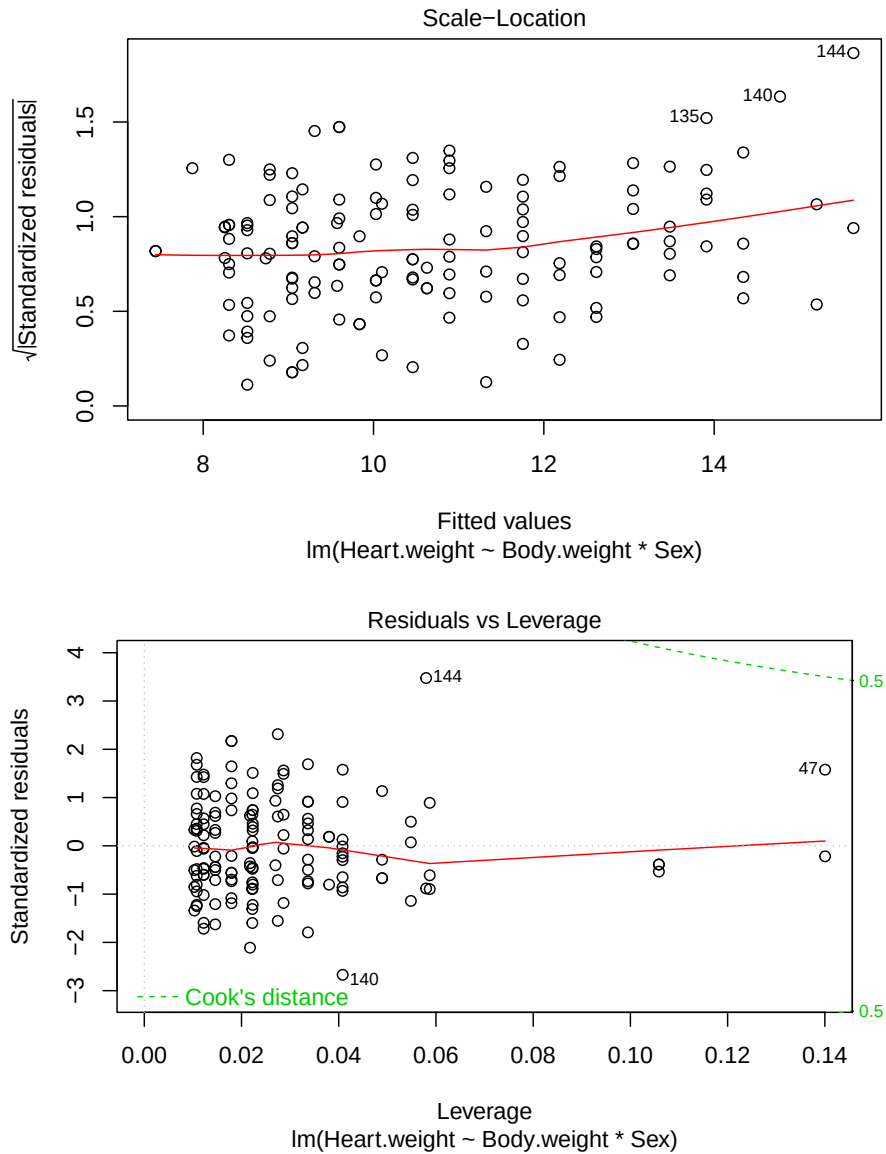
1.6 The *plot.lm()* function

Above we create all the graphs needed for the residual analysis by ourselves. However, there is a function that does that for you. The *plot.lm()* function, called via the generic function *plot()* produce the four most important plots required for a residual analysis of a linear model.

```
plot(lm.cats)
```

²Note that that the *plot()* method function here is calling the *plot.lm* function. Type *?stats::plot.lm* to get more information.





This function is very handy, as it produces in one go all four plots. However, sometimes we may want to customise our plots or create additional ones.

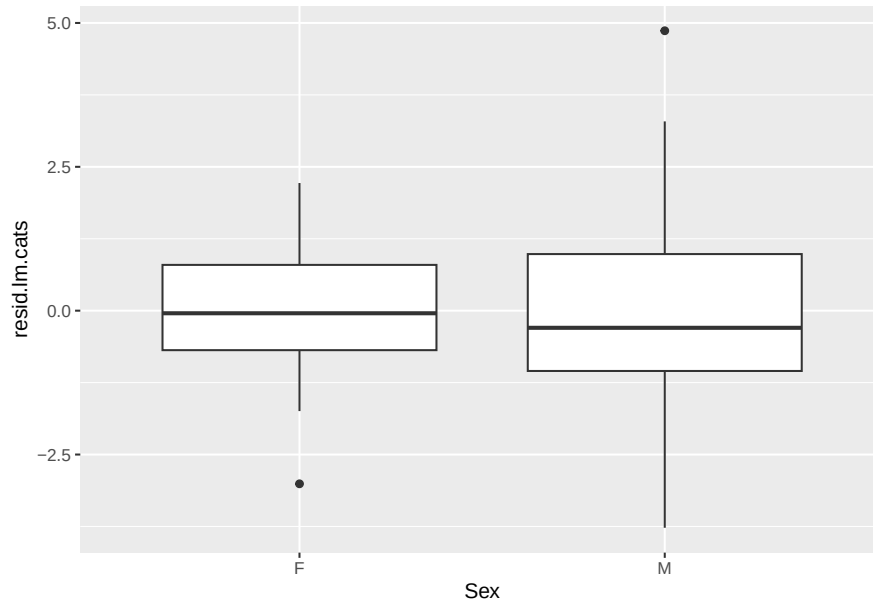
For example here, we may want to further test the linearity of the effect for *Body.weight* in the females group.

1.7 Further plots (*)

It is good practice to plot the residuals against all predictors in a model.

We start with the *Sex* predictor.

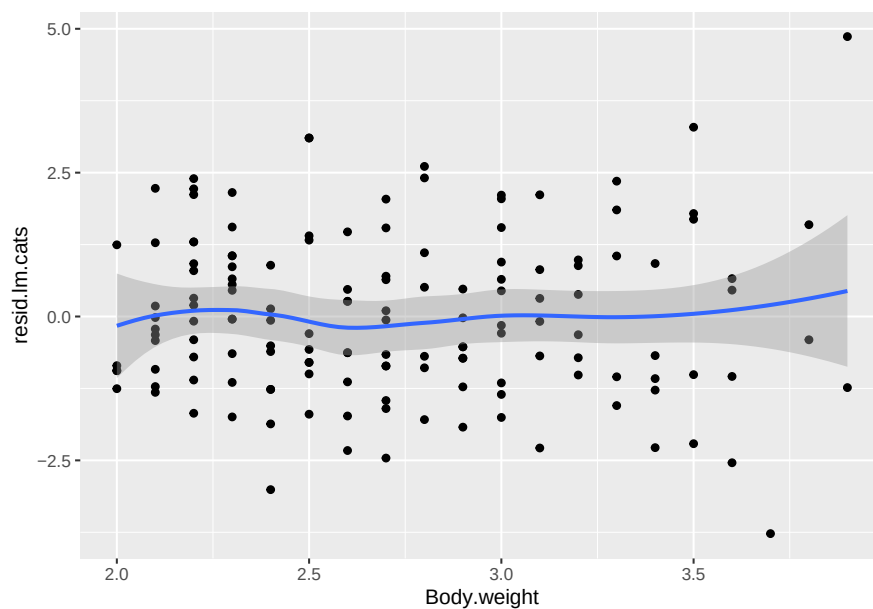
```
ggplot(data = d.cats,
       mapping = aes(y = resid.lm.cats,
                     x = Sex)) +
  geom_boxplot()
```



This graph can help understand whether there are issues with the variability of the residuals (i.e. the homoscedasticity assumption). We already tested this assumption with the “scale-location plot” and here we can confirm that the variability of residuals seem to be constant (even across groups). More information about the cases where the homoscedasticity assumption is not fulfilled can be found in the “Heteroscedasticity” subsection in the Appendix.

We now plot the residuals against the body weight predictor.

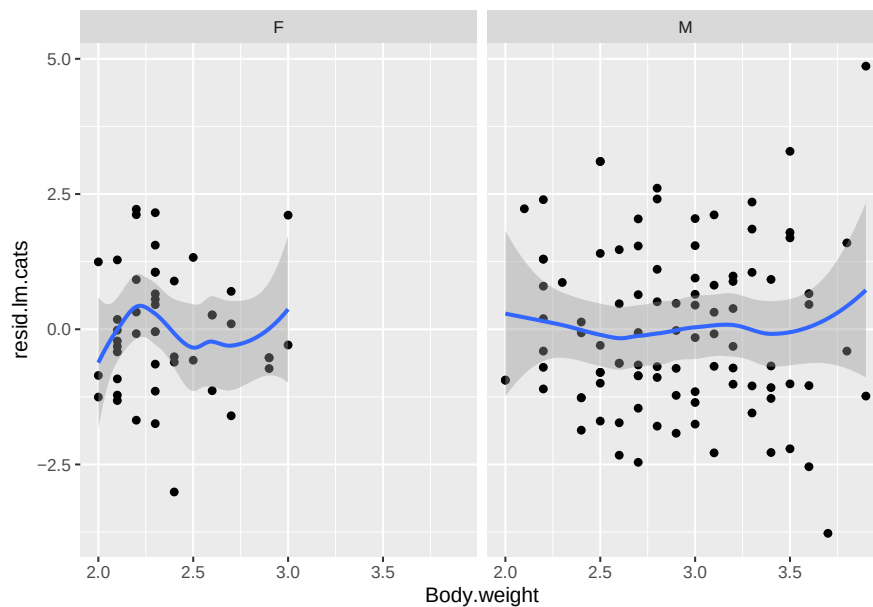
```
ggplot(data = d.cats,
       mapping = aes(y = resid.lm.cats,
                     x = Body.weight)) +
  geom_point() +
  geom_smooth()
```



The smoother seems to be flat on zero. This must not come as a surprise because, in this very simple example with only two predictors, the Tukey-Anscombe plot is very similar to this plot.

Nevertheless, you may remember that there were indications that the effect of *Body.weight* for females may be non-linear. Therefore, we may want to inspect this a bit further.

```
ggplot(data = d.cats,
       mapping = aes(y = resid.lm.cats,
                     x = Body.weight)) +
  geom_point() +
  geom_smooth() +
  facet_wrap(. ~ Sex)
```



The effect of *Body.weight* does not seem to be perfectly linear for females. However, this may be due “natural randomness” of the data. If we would like to inspect this potential non-linearity further we could use Generalised Additive Models, for short, that drop the linearity assumption. Note that smoothers are indeed GAMs.

2 Appendix

2.1 Heteroscedasticity (**)

Let's assume that by looking at the scale-location plot (absolute residuals versus fitted values) we note a problem of non-constant variance of the residuals (i.e. heteroscedasticity). There are three main solutions to this issue.

1. The response variable needs to be transformed such that the variance of the residuals is stabilised.
2. The model must be updated such that a predictor indicates that residuals have different variances (i.e. weighted regression).
3. An important predictor or interaction is missing from the model. The model must be updated.

1. Transforming the response variable

It is common practice to apply the following transformation to the response variable. These transformation are known as “first aid transformations”.

- $\log(y)$ for amounts. Amounts are variables that can only take positive values such as salary, weight or concentrations.
- square-root transformation for counts. Count data are positive integers (e.g. number of siblings). Note also that there is a wide range of extensions of the linear model that were developed to deal with count data (e.g. Poisson model).
- arc-sine-square-root transformation for percentages³.

1. Weighted regression

Sometimes, the variance of the residual can happen to depend on one predictor only. For example, it may be very different between groups (say males and females). The variance of the residuals can also depend on a continuous predictor (e.g. older cats show more variable observations).

In this case, weighted regression can be used. It consist in defining a weight for each observation that is inversely proportional to its variance. More variable observations are less reliable and have therefore a lower weight.

A classical example where weighted regression is very helpful is when the observations themselves represent averages of other observations. Let’s assume that we are analysing the factors that influence body weight in the Swiss population. For privacy reasons the data was aggregated at municipality level, by averaging the weight of all measured people in a given municipality. Obviously, the averages for a very small municipality (say Vergeletto in Valle Onsernone) is based on a few measurements and it thus less precise than the average for Zurich that is based on way more observations. In this case, we include the “sample size” information in our model via the weights argument as `lm(average.weight ~ ., weights = N)`, where N indicated the number of observations taken to estimate averages in each municipality.

2.2 Tests to test assumptions (**)

There are two main reason why tests should not be used to test assumptions about the errors of a linear model.

1. Philosophical reason: these tests also rely on some assumptions. . . which should also be tested. . . leading to a infinite circle of testing.
2. Practical reason: the power of any test relies on the sample size⁴ . For low sample size the test could be non-significant even though the residuals are, for example, clearly not normally distributed. On the other hand a very small deviation from normality, may be detected in a very large sample size.

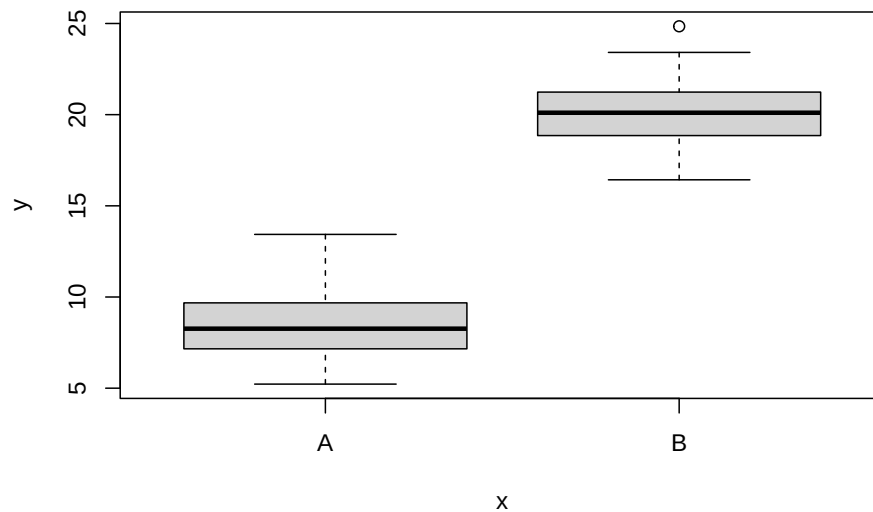
2.3 Normality of the data

A commonly encountered mistake is to check the normality of the response variable itself. To explain why this is incorrect we simulate a very simple example.

³In **R** this is achieved with `asin(sqrt(y))` where y is a variable bounded between 0 and 1.

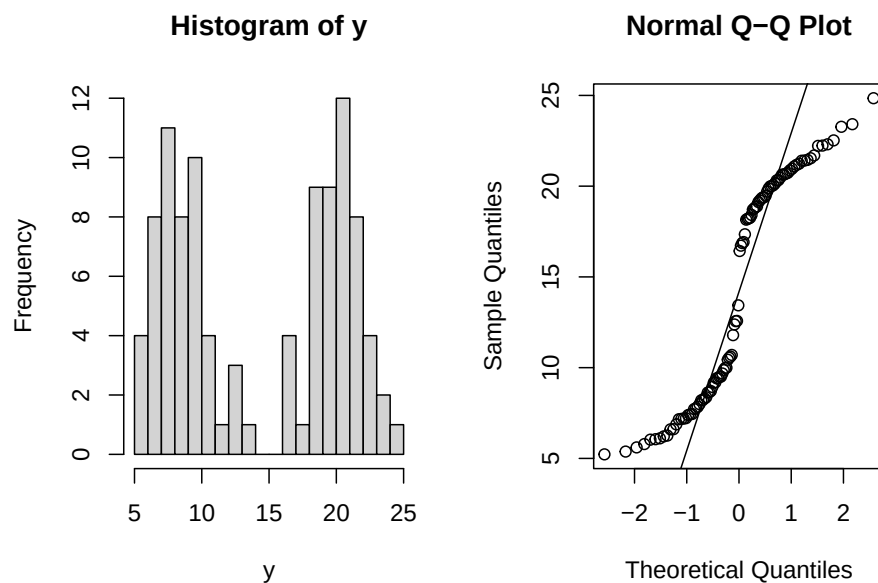
⁴In statistics the power is defined as the probability to detect an effect when this is present. In more formal words, the probability to reject the null hypothesis when this is false.


```
set.seed(7)
x <- gl(n = 2, k = 50, labels = c("A", "B"))
errors <- rnorm(n = 100, sd = 2)
y <- rep(x = c(8,20), each = 50) + errors
plot(y ~ x)
```



The errors were simulated from a normal distribution, however, if we were to produce an histogram of the response variable itself, it would obviously not look normal.

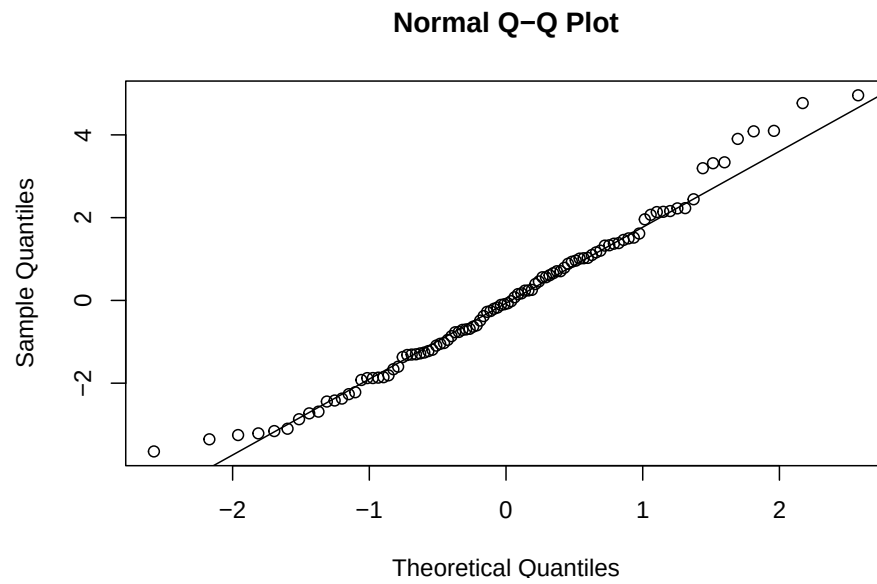
```
par(mfrow = c(1,2))
hist(y, breaks = 20)
qqnorm(y)
qqline(y)
```



Indeed, the two groups are well visible in the histogram as well as in the the QQ-plot.

However, if we were to look at the residuals, they would appear to be consistent with the normal distribution.

```
lm.sim <- lm(y ~ x)
qqnorm(resid(lm.sim))
qqline(resid(lm.sim))
```



3 Session Information

```
sessionInfo()
```

```
R version 4.5.1 (2025-06-13)
```

```
Platform: aarch64-apple-darwin20
```

```
Running under: macOS Sequoia 15.6.1
```

```
Matrix products: default
```

```
BLAS: /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRblas.0.dylib
```

```
LAPACK: /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRlapack.dylib; LAPACK vers
```

```
locale:
```

```
[1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8
```

```
time zone: Europe/Zurich
```

```
tzcode source: internal
```

```
attached base packages:
```

```
[1] stats      graphics  grDevices  utils      datasets  methods    base
```

```
other attached packages:
```

```
[1] plgraphics_1.3 ggplot2_3.5.2 knitr_1.50
```

```
loaded via a namespace (and not attached):
```

[1] Matrix_1.7-3	gtable_0.3.6	dplyr_1.1.4	compiler_4.5.1
[5] Rcpp_1.1.0	tidyselect_1.2.1	tinytex_0.57	splines_4.5.1
[9] scales_1.4.0	boot_1.3-31	yaml_2.3.10	fastmap_1.2.0
[13] lattice_0.22-7	R6_2.6.1	labeling_0.4.3	generics_0.1.4
[17] rbibutils_2.3	MASS_7.3-65	tibble_3.3.0	nloptr_2.2.1
[21] minqa_1.2.8	pillar_1.11.0	RColorBrewer_1.1-3	rlang_1.1.6
[25] xfun_0.52	cli_3.6.5	withr_3.0.2	magrittr_2.0.3
[29] mgcv_1.9-3	Rdpack_2.6.4	digest_0.6.37	grid_4.5.1
[33] rstudioapi_0.17.1	lme4_1.1-37	lifecycle_1.0.4	nlme_3.1-168
[37] reformulas_0.4.1	vctrs_0.6.5	evaluate_1.0.4	glue_1.8.0
[41] farver_2.1.2	survival_3.8-3	rmarkdown_2.29	chron_2.3-62
[45] tools_4.5.1	pkgconfig_2.0.3	htmltools_0.5.8.1	